FitCore Gym Management System



Project Overview

1. **Introduction**
2. **Project Aims and Objectives**
3. **System Overview**
4. **Technology Stack**
5. **Core Entities and Relationships**
6. **Key Features**
7. **System Architecture**
8. **Project Team and Roles**
9. **Conclusion**

Introduction:

FitCore Gym is a longstanding and popular gym in the Sunnyview community. Its current problem is that the business currently manages its members, personal trainers, training programmes, workout plans, and workout tasks using manual, paper-based processes. In recent months as the gym has received an influx of new members, the manual approach has become inefficient, error-prone, and difficult to scale.

The **FitCore Gym Management System** is a full-stack web application designed to digitalise these operations. Providing a single, centralised platform where administrators, personal trainers, and gym members can interact with data relevant to their respective roles, securely and in real time.

Project Aim

To design a user-friendly full stack Gym Management System centralising the management of gym members, personal trainers, training programmes, workout plans, and workout tasks on a single secure platform.

Objectives

- Implement a relational database using Entity Framework Core (Code-First) with Microsoft SQL Server.
- Provide role-based authentication and authorisation using ASP.NET Identity with JWT tokens.
- Build a RESTful Web API using ASP.NET Core 8 that exposes endpoints for all core entities.
- Develop a React frontend styled with React-Bootstrap that provides role-specific dashboards.
- Implement CRUD operations, search functionality, and filtering for the required entities.
- Demonstrate correct implementation of one-to-many relationships between entities.
- Document the system with an ERD, Use Case Diagram, and a research section covering key technical decisions.

System Overview

The FitCore Gym Management System has three user roles, each with a distinct set of permissions and responsibilities:

Role	Primary Responsibilities
Administrator	Manages members, trainers, and training programmes; assigns trainers to members; assigns members to programmes; searches and lists users by type.
Personal Trainer	Views assigned members and their programmes; creates and manages workout plans and workout tasks.
Gym Member	Views their assigned programme, workout plans, and tasks; updates the status of workout tasks; filters tasks by status.

- All three roles interact on one platform, but each sees only the data and functionality appropriate to their role. This is enforced at both the API level (through **[Authorize (Roles = "...")]**) and the frontend level (through protected routes).

Technology Stack

Backend

- ASP.NET Core 8 Web API — RESTful API framework
- Entity Framework Core (Code-First) — ORM for database access
- Microsoft SQL Server — relational database
- ASP.NET Identity — user management and password hashing
- JWT Bearer Tokens — stateless authentication
- Data Annotations — server-side validation

Frontend

- React — component-based UI library
- React-Bootstrap — UI component library
- React Router — client-side routing
- Axios — HTTP client with JWT interceptor
- JavaScript (ES6+) — no TypeScript

Development Tools

- Git and GitHub — version control
- Swagger / Postman — API testing
- Visual Studio Code — development environment
- Trello / Planner — task tracking

Core Entities and Relationships

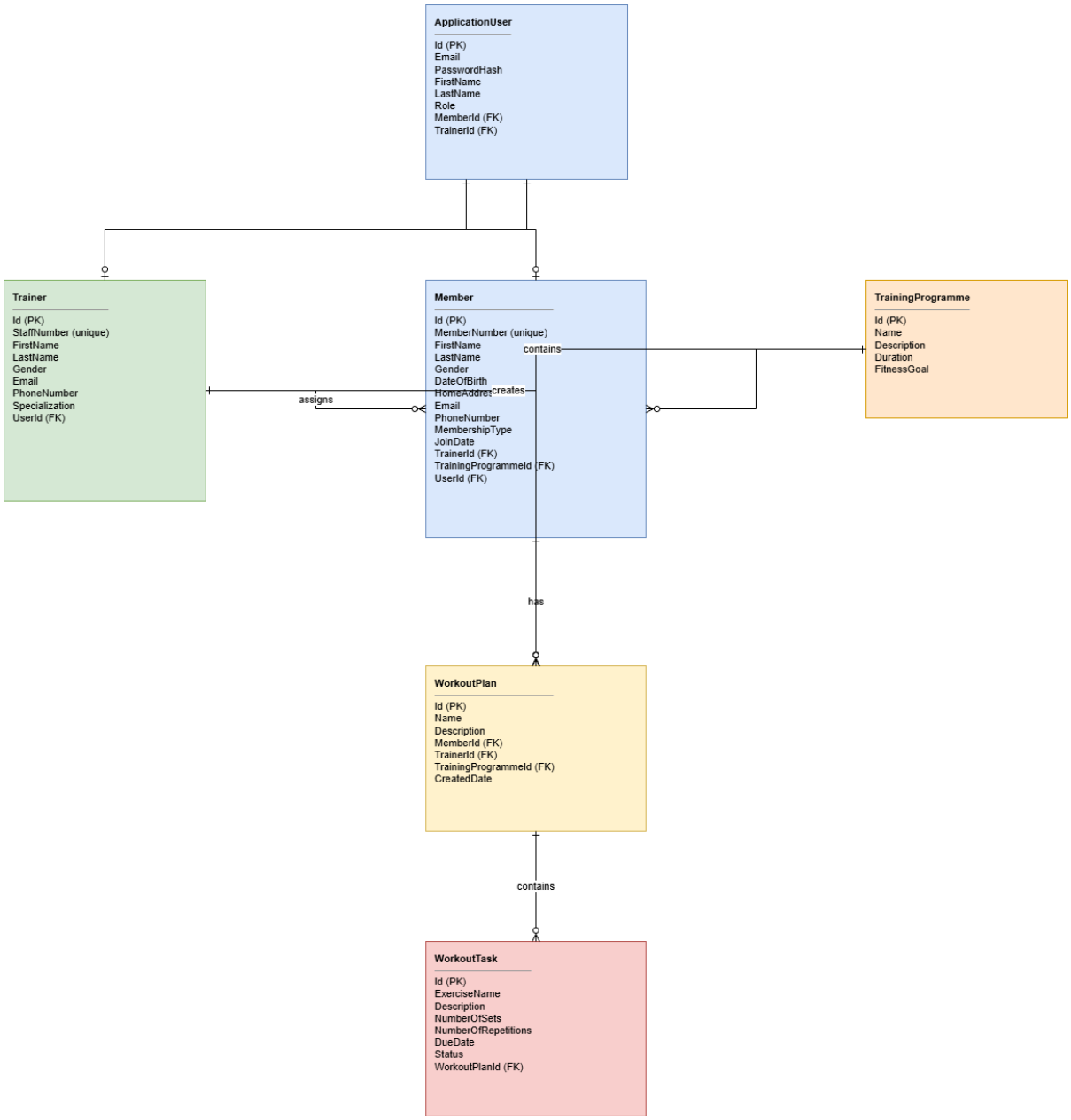
The system is built around six core entities:

Entity	Purpose
ApplicationUser	Identity user (extends IdentityUser) — represents anyone who logs in
Member	Gym member record with personal, contact, and membership details
Trainer	Personal trainer record with specialisation details
TrainingProgramme	Programme with name, duration, and fitness goal
WorkoutPlan	A plan belonging to a member, created by a trainer, linked to a programme
WorkoutTask	An exercise inside a workout plan (sets, reps, due date, status)

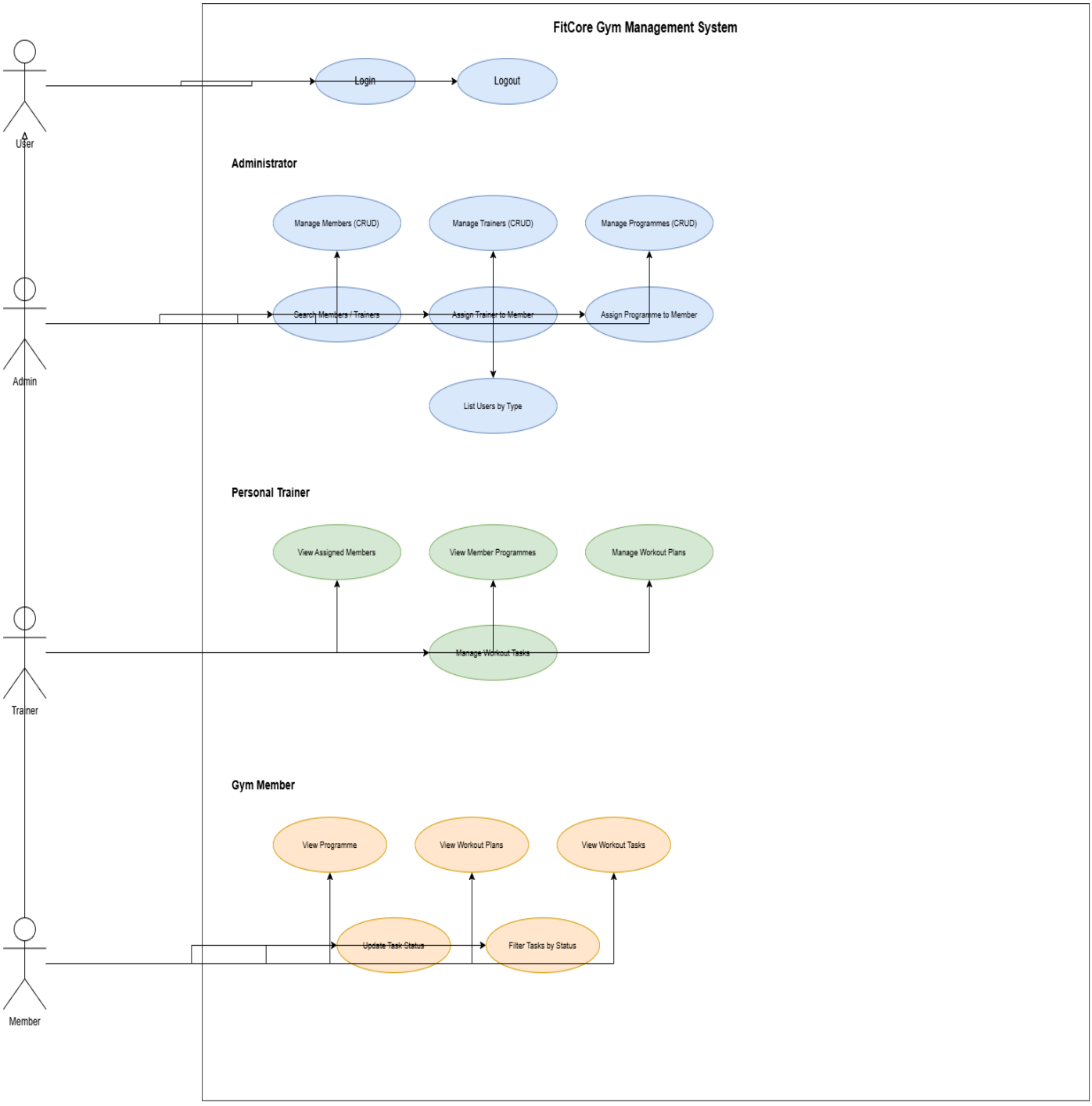
Relationships

- The following one-to-many relationships are enforced by EF Core:
- **Trainer 1** — *** Member** — one trainer can be assigned to many members
- **TrainingProgramme 1** — *** Member** — one programme can be assigned to many members
- **TrainingProgramme 1** — *** WorkoutPlan** — one programme can contain many workout plans
- **Member 1** — *** WorkoutPlan** — one member can have many workout plans
- **Trainer 1** — *** WorkoutPlan** — one trainer can create many workout plans
- **WorkoutPlan 1** — *** WorkoutTask** — one plan can contain many tasks
- In addition:
- **ApplicationUser 1** — **0..1 Member**
- **ApplicationUser 1** — **0..1 Trainer**
- Full details are documented in the Entity Relationship Diagram (ERD) section

ERD


ApplicationUser
Id (PK)
Email
PasswordHash
FirstName
LastName
Role
MemberId (FK)
TrainerId (FK)
Trainer
Id (PK)
StaffNumber (unique)
FirstName
LastName
Gender
Email
PhoneNumber
Specialization
UserId (FK)
assigns
Member
Id (PK)
MemberNumber (unique)
FirstName
LastName
Gender
DateOfBirth
HomeAddress
Email
PhoneNumber
MembershipType
JoinDate
TrainerId (FK)
TrainingProgrammeId (FK)
UserId (FK)
contains
creates
TrainingProgramme
Id (PK)
Name
Description
Duration
FitnessGoal
has
WorkoutPlan
Id (PK)
Name
Description
MemberId (FK)
TrainerId (FK)
TrainingProgrammeId (FK)
CreatedDate
contains
WorkoutTask
Id (PK)
ExerciseName
Description
NumberOfSets
NumberOfRepetitions
DueDate
Status
WorkoutPlanId (FK)

Use Case Diagram



Key Features

Authentication and Authorisation

- Secure login with email and password using ASP.NET Identity
- JWT tokens issued on login, valid for 60 minutes
- Role-based access control **(Admin, Trainer, Member)**
- All API endpoints protected with **[Authorize(Roles = "...")]**
- Frontend routes wrapped in **ProtectedRoute** components

Member Management (Admin)

- Full CRUD operations on gym members
- Search by member number, name, or surname
- Assign a trainer to a member
- Assign a training programme to a member

Trainer Management (Admin)

- Full CRUD operations on trainers
- Search by staff number, name, or surname

Training Programme Management (Admin)

- Full CRUD operations on programmes
- Each programme has a name, description, duration, and fitness goal

Workout Plans (Trainer)

- Create and manage workout plans for assigned members
- Each plan is linked to a member, a trainer, and a training programme

Workout Tasks (Trainer and Member)

- Trainers create workout tasks within a plan (exercise name, sets, reps, due date)
- Members view their tasks and update status: **NotStarted**, **InProgress**, **Complete**
- Members can filter their tasks by status

Search and Filtering

- Server-side search using EF Core Where and Contains queries
- Status-based filtering for workout tasks
- Results returned as DTOs for consistency and security

Validation

- Server-side validation using Data Annotations **([Required], [StringLength], [Range]**, **[EmailAddress])**
- Consistent error responses across the API
- Client-side validation mirrored in React forms

System Architecture

The system follows a **three-tier architecture**:

Tier 1 — Presentation (Frontend)

React single-page application running in the browser. Communicates with the API using Axios over HTTPS. Stores JWT and user info in **localStorage**.

Tier 2 — Application (Backend)

ASP.NET Core 8 Web API. Handles HTTP requests, validates input, applies role-based authorisation, and interacts with the database through EF Core.

Tier 3 — Data (Database)

Microsoft SQL Server database, generated and maintained through EF Core migrations (Code-First). Holds all entities and relationships.

Request Flow (example)

1. User logs in → frontend sends **POST /api/auth/login** with email and password.
2. Backend validates credentials against ASP.NET Identity.
3. Backend returns a JWT token and user info.
4. Frontend stores the token in **localStorage**.
5. Every subsequent request includes **Authorization: Bearer <token>**.
6. Backend validates the token, checks the role, and processes the request.
7. Backend returns JSON; frontend renders the data.

Conclusion

The FitCore Gym Management System demonstrates the design and implementation of a real-world, full-stack web application. It combines a secure, role-based backend with a modern, responsive frontend, and it addresses the operational needs of a growing gym.

- The project showcases practical skills in:
- Entity Framework Core (Code-First) and relational database design
- ASP.NET Identity and JWT-based authentication
- RESTful API design with role-based authorisation
- React with React-Bootstrap for a professional user interface
- Search, filtering, and validation
- Version control and team collaboration

When complete, the system will allow FitCore Gym to replace its manual processes with a reliable, secure, scalable and user-friendly digital platform.

Research Section: React, ASP.NET Core Web API Communication

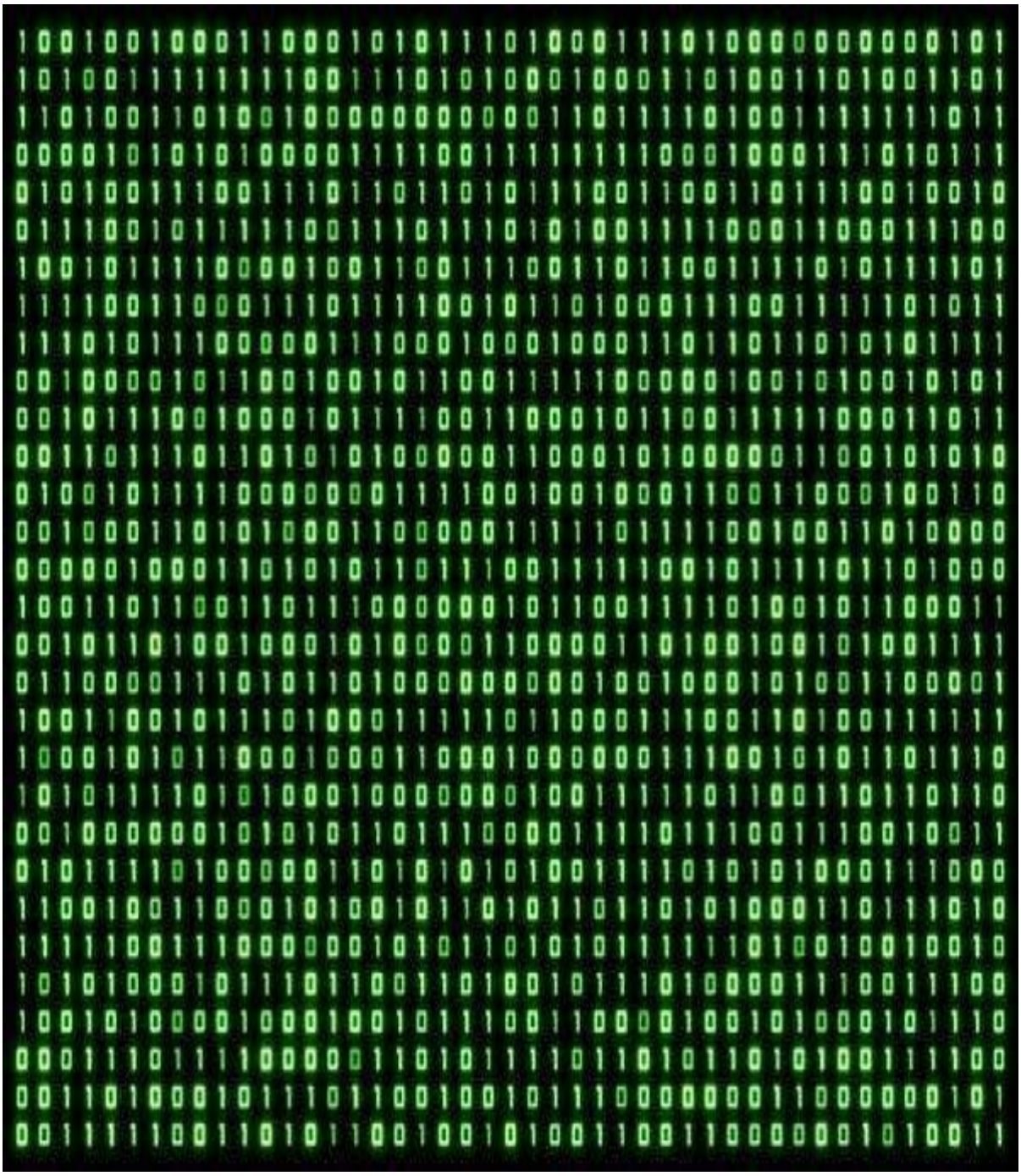


Table of Contents

Research Section: React, ASP.NET Core Web API Communication 13

Introduction 15

Axios 16

API Configuration 17

Authentication Flow 18

Sending Requests — The Service Layer 19

Receiving Responses 20

Error Handling 22

Cross-Origin Resource Sharing (CORS) 23

ASP.NET Identity and Role-Based Authorization 24

Summary 28

Introduction

The FitCore Gym Management System is a full-stack web application built on two distinct technologies: a React single-page application running in the browser, and an ASP.NET Core 8 Web API running on the server. These two are used to communicate reliably, securely, and efficiently. This research section explains how that communication is implemented

React and ASP.NET Core are separate applications, running on different ports during development, they do not share memory. Every interaction between them must happen over HTTP using JSON as the data format, with authentication carried by JWT tokens on every request.

This section covers:

- The choice of HTTP client (Axios)
- How requests and responses are structured
- How authentication is attached to every request
- How errors are handled consistently
- How Cross-Origin Resource Sharing (CORS) is configured

Axios

Because react does not include a built-in HTTP client a browser would provide native API's, we specifically chose Axios over the native API for the following reasons:

- Cleaner syntax — Axios automatically parses JSON responses, so there's no need to call **response.Json()** on every request.
- Interceptors — Axios allows us to define functions that run before every request or after every response. This is essential for attaching JWT tokens automatically.
- Better error handling — Axios rejects promises on non-2xx HTTP status codes by default, making error handling more predictable.
- Wide adoption — Axios is the de facto standard in the React community, so documentation and examples are abundant

API Configuration

Other than configuring Axios separately in every file, we create a **single Axios instance** in frontend/src/services/api.js. All services import this instance. This ensures:

- One place to set the base URL
- One place to attach authentication
- One place to add global error handling
- Consistency across the entire application

```
import axios from 'axios';
const api = axios.create({
  baseURL: 'https://localhost:7001/api',
});
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});
export default api;
```

Explanation

- **axios.create() —** creates a reusable instance with shared configuration. Every request made through this instance automatically uses baseURL, so service files can call **api.get('/members')** instead of the full URL.
- **interceptors.request.use(...) —** a function that runs **before** every request is sent. It reads the JWT from localStorage and attaches it as an Authorization header. This means no service file ever has to remember to add the token — it happens automatically.

Authentication Flow

Step 1 — Login

The frontend sends a POST request **to /api/auth/login** with the user's email and password:

```
const response = await api.post('/auth/login', { email, password });
```

Step 2 — Token issued

The API validates the credentials against ASP.NET Identity. If valid, it returns a JSON response containing a JWT token and user information:

```
{
  "token": "eyJhbGciOiJIUzI1NiIs...",
  "user": {
    "id": "abc-123",
    "email": "member@fitcore.co.za",
    "role": "Member",
    "memberId": 1,
    "trainerId": null,
    "firstName": "Test",
    "lastName": "Member"
  }
}
```

Step 3 — Token stored

The frontend stores both the token and the user object in localStorage:

```
localStorage.setItem("token", token);
localStorage.setItem("user", JSON.stringify(user));
```

Step 4 — Token attached to every subsequent request

Because of the Axios request interceptor, the Authorization: Bearer <token> header is automatically added to every request from that point onward.

Step 5 — Backend validates

The API's [Authorize] attribute validates the token and extracts the user's role from the JWT claims. If the token is invalid or expired, the API returns 401 Unauthorized. If the user's role does not match the required role, the API returns 403 Forbidden.

Step 6 — Logout

On logout, the frontend removes both keys from **localStorage**. Because the token is no longer present, the interceptor stops adding the header, and protected endpoints become inaccessible.

Sending Requests — The Service Layer

All API calls are wrapped in service files under **frontend/src/services/**. Each service exposes functions that match the API contract exactly. This layer isolates the rest of the frontend from the details of HTTP.

Example — from workoutTaskService.js:

```
const workoutTaskService = {
  getByPlan: async (planId, status) => {
    const params = status ? { status } : {};
    const response = await api.get(`/workouttasks/plan/${planId}`, { params });
    return response.data;
  },

  updateStatus: async (id, status) => {
    const response = await api.patch(`/workouttasks/${id}/status`, { status });
    return response.data;
  },
};
```

- **api.get()** sends an HTTP GET request.
- Path parameters are inserted directly into the URL, e.g. **/workouttasks/${id}/status.**
- Query parameters are passed as a params object. Axios converts them into ?status=InProgress automatically.
- **response.data** contains the parsed JSON body returned by the API.
- This pattern is repeated in every service: memberService.js, trainerService.js, programmeService.js, workoutPlanService.js, and workoutTaskService.js.

Receiving Responses

The API always returns **JSON**. ASP.NET Core serialises C# objects into JSON automatically and uses **camelCase** for property names (e.g. **firstName, memberNumber, trainingProgrammeId**). This matches JavaScript conventions, so the frontend can consume the data directly without any transformation.

C# DTO:

```
public class MemberDto
{
    public int Id { get; set; }
    public string MemberNumber { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
}
```

returned to the frontend:

```
{
  "id": 1,
  "memberNumber": "M0001",
  "firstName": "Test",
  "lastName": "Member"
}
```

React page then binds these properties directly into the UI:

```
<td>{member.memberNumber}</td>
<td>{member.firstName} {member.lastName}</td>
```

Error Handling

A consistent error format is essential so that the frontend can display user-friendly messages. The API uses a standard error shape for all failures:

```
{
  "status": 400,
  "message": "Validation failed",
  "errors": {
    "email": ["Email is required"],
    "numberOfSets": ["Number of sets must be between 1 and 20"]
  }
}
```

On the frontend, every service call is wrapped in a **try/catch** block. If the request fails, the error is caught and displayed to the user:

```
try {
  const data = await memberService.getMyTasks(statusFilter);
  setTasks(data);
} catch (err) {
  setError(err.response?.data?.message || 'Could not load tasks.');
}
```

- **try** — runs the request.
- **catch (err)** — runs if the request fails (network error or non-2xx status).
- **err.response?.data?.message** — extracts the human-readable message from the API. The optional chaining (?.) prevents a crash if **err.response** is undefined (e.g. network failure).
- **Fallback message** — if no message is available, a generic one is used.

For form validation, the errors object is used to attach specific messages to specific fields

```
setFormError(err.response?.data?.errors?.email?.[0] || 'Could not save member.');
```

Cross-Origin Resource Sharing (CORS)

During development, React runs on **http://localhost:3000** and the API runs on **https://localhost:7001**. Browsers block cross-origin requests by default for security reasons. To allow the frontend to reach the API, CORS must be configured on the backend.

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowReactApp", policy =>
    {
        policy.WithOrigins("http://localhost:3000")
              .AllowAnyHeader()
              .AllowAnyMethod()
              .AllowCredentials();
    });
});

// Later in the pipeline
app.UseCors("AllowReactApp");
```

WithOrigins("http://localhost:3000") — explicitly allows the React development server. In production, this would be replaced with the deployed frontend URL.

AllowAnyHeader() — permits the Authorization and Content-Type headers.

AllowAnyMethod() — allows GET, POST, PUT, PATCH, and DELETE.

AllowCredentials() — required if cookies or authorization headers are sent.

Without this configuration, the browser would block every request from React with a CORS error before it even reaches the API.

ASP.NET Identity and Role-Based Authorization

Authentication and authorization are handled by ASP.NET Identity combined with JWT Bearer tokens. This provides a stateless, scalable authentication mechanism suitable for a RESTful API.

ASP.NET Identity manages users, passwords, and roles. The **ApplicationUser** class extends **IdentityUser**:

```
public class ApplicationUser : IdentityUser
{
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public int? MemberId { get; set; }
    public int? TrainerId { get; set; }
}
```

roles are seeded at startup: **Admin**, **Trainer**, and **Member**.

JWT Token Generation

On successful login, the API generates a JWT containing claims for the user's **ID**, **email**, **role**, and associated entity IDs:

```
var claims = new List<Claim>
{
    new Claim(ClaimTypes.NameIdentifier, user.Id),
    new Claim(ClaimTypes.Email, user.Email),
    new Claim(ClaimTypes.Role, role),
    new Claim("memberId", user.MemberId?.ToString() ?? ""),
    new Claim("trainerId", user.TrainerId?.ToString() ?? "")
};

var token = new JwtSecurityToken(
    issuer: _config["Jwt:Issuer"],
    audience: _config["Jwt:Audience"],
    claims: claims,
    expires: DateTime.UtcNow.AddMinutes(60),
    signingCredentials: credentials
);
```

Protecting Endpoints

```
[Authorize(Roles = "Admin")]
[HttpPost("members")]
public async Task<IActionResult> CreateMember(CreateMemberDto dto) { ... }

[Authorize(Roles = "Trainer,Admin")]
[HttpGet("workoutplans")]
public async Task<IActionResult> GetWorkoutPlans() { ... }
```

- If no valid token is provided → **401 Unauthorized**
- If the token is valid but the role does not match → **403 Forbidden**

Frontend Role Enforcement

The frontend uses a **ProtectedRoute** component that checks the user's role from **localStorage** before providing a route. If the user lacks the required role, they are redirected to an "Unauthorized" page. This provides a better user experience but is not a security measure. Only the API enforces the real authorization.

Search Functionality

Backend Implementation

Search is implemented server-side using Entity Framework Core's Where and Contains methods. For example, searching members by name, surname, or member number:

```
[HttpGet("search")]
public async Task<IActionResult> SearchMembers([FromQuery] string query)
{
    var results = await _context.Members
        .Where(m => m.FirstName.Contains(query) ||
                    m.LastName.Contains(query) ||
                    m.MemberNumber.Contains(query))
        .Select(m => new MemberDto
        {
            Id = m.Id,
            MemberNumber = m.MemberNumber,
            FirstName = m.FirstName,
            LastName = m.LastName
        })
        .ToListAsync();

    return Ok(results);
}
```

Frontend Implementation

The frontend sends the search term as a query parameter and displays the results:

```
const searchMembers = async (query) => {
  const response = await api.get('/members/search', { params: { query } });
  return response.data;
};
```

Entity Framework Core Relationships

One-to-Many Relationships

The system implements the following one-to-many relationships using EF Core Code-First:

Relationship	Description
Trainer → Members	One trainer can be assigned to many members
TrainingProgramme → Members	One programme can be assigned to many members
TrainingProgramme → WorkoutPlans	One programme can contain many workout plans
Member → WorkoutPlans	One member can have many workout plans
Trainer → WorkoutPlans	One trainer can create many workout plans
WorkoutPlan → WorkoutTasks	One plan can contain many tasks

Example — WorkoutPlan and WorkoutTask:

```
public class WorkoutPlan
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int MemberId { get; set; }
    public Member Member { get; set; }
    public int TrainerId { get; set; }
    public Trainer Trainer { get; set; }
    public int TrainingProgrammeId { get; set; }
    public TrainingProgramme TrainingProgramme { get; set; }
    public ICollection<WorkoutTask> Tasks { get; set; }
}

public class WorkoutTask
{
    public int Id { get; set; }
    public string ExerciseName { get; set; }
    public int NumberOfSets { get; set; }
    public int NumberOfReps { get; set; }
    public DateTime DueDate { get; set; }
    public string Status { get; set; }
    public int WorkoutPlanId { get; set; }
    public WorkoutPlan WorkoutPlan { get; set; }
}
```

Configuring Relationships in DbContext

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<WorkoutPlan>()
        .HasOne(wp => wp.Member)
        .WithMany(m => m.WorkoutPlans)
        .HasForeignKey(wp => wp.MemberId)
        .OnDelete(DeleteBehavior.Restrict);

    modelBuilder.Entity<WorkoutTask>()
        .HasOne(wt => wt.WorkoutPlan)
        .WithMany(wp => wp.Tasks)
        .HasForeignKey(wt => wt.WorkoutPlanId)
        .OnDelete(DeleteBehavior.Cascade);
}
```

One-to-Zero-or-One Relationships

ApplicationUser has optional one-to-zero-or-one relationships with **Member** and **Trainer**

```
modelBuilder.Entity<ApplicationUser>()
    .HasOne(u => u.Member)
    .WithOne()
    .HasForeignKey<ApplicationUser>(u => u.MemberId);

modelBuilder.Entity<ApplicationUser>()
    .HasOne(u => u.Trainer)
    .WithOne()
    .HasForeignKey<ApplicationUser>(u => u.TrainerId);
```

Migrations

EF Core Migrations are used to create and update the database schema:

```
dotnet ef migrations add InitialCreate
dotnet ef database update
```

Summary

This research summary documents the key technical and implementational decisions used to enable the FitCore Gym Management System's React frontend to communicate securely and efficiently with its ASP.NET Core 8 Web API backend. Axios with interceptors provides automatic JWT attachment, the service layer isolates HTTP concerns, consistent error handling ensures a good user experience, and CORS configuration allows cross-origin requests during development. Combined with ASP.NET Identity for authentication, role-based authorization for security, EF Core for data relationships, and server-side search functionality, our system provides a robust, and maintainable full-stack solution, which can also be easily scaled to meet increasing member numbers at FitCore Gym